GH raisoni COLLEGE | Engineering and Management Pune

An Empowered Autonomous Institute, Affiliated to SPPU, Pune
NAAC Accredited "A+" Grade & NBA

Department of AI & AIML Engineering

LAB MANUAL

Subject: DATA SCIENCE FOR ENGINEERS(UCAMP401D)

Class: BTECH (CSE AIML AB – 2023 Course)

Examination Scheme:
Practical (INT): 25 Marks
Total: 25 Marks

Prepared By: Dr.Shinde B.A

Institute Vision & Mission

Vision

To achieve excellent standards of quality education by keeping pace with rapidly changing technologies and to create technical manpower of global standards with capabilities of accepting new challenges

Mission

Our efforts are dedicated to impart quality and value-based education to raise satisfaction level of all stakeholders. Our strength is directed to create competent professionals. Our endeavor is to provide all possible support to promote research and development activities.

Department Vision & Mission

VISION:

To develop globally competent professionals with a strong foundation in Artificial Intelligence, empowering them to drive innovation and positively impact industry and society.

MISSION:

The department continuously strives to:

M1: Provide a comprehensive, cutting-edge curriculum that keeps pace with the rapid advancements in AI technologies by collaborating with leading companies.

M2: Nurture a diverse, globally-minded student community by providing practical, experiential learning opportunities for students to apply their knowledge and skills to solve real-world problems.

M3: Create a culture that promotes continuous learning, research, and entrepreneurial skills in order to give our students a lifelong passion for staying abreast of new developments

LIST OF EXPERIMENTS

Sr. No.	Title of Experiment	CO
1	Python code to get information about the file pertaining to the file mode and to get time values with components using local time and gm time.	CO1
2	Perform Array operations using Numpy: Write a Python program to i. Create N-dim arrays ii. Use inbuilt function (zeros, ones, full, eye) to create array iii. Use shape, reshape, transpose function iv. Use sort function and Sorting along axes v. Use arange, linspace, logspace function vi. Array Indexing , slicing, Joining, Splitting, and Searching arrays vii. Statistical Functions on arrays	CO1
3	Perform Data Manipulation operations using Pandas package on dataframes.	CO2
4	Apply and explore multiple types of plots using Matplotlib package	CO2
5	Write a Python Program to import a CSV file and perform various Statistical and Comparison operations on rows/columns.	CO2
6	Write a Python Program to import any CSV file to Pandas DataFrame and perform the following: i. Visualize the first and last 10 records ii. Get the shape, index and column details iii. Select/Delete the records(rows)/columns based on conditions. iv. Perform ranking and sorting operations. v. Do required statistical operations on the given columns. vi. Find the count and uniqueness of the given categorical values. vii. Rename single/multiple columns.	CO3
7	Write a Python Program to import any CSV file to Pandas DataFrame and perform the following: i. Handle missing data by detecting and dropping/ filling missing values. ii. Transform data using different methods. iii. Detect and filter outliers. iv. Perform Vectorized String operations on Pandas Series. v. Visualize data using Line Plots, Bar Plots, Histograms, Density Plots and Scatter Plots.	CO2

8	Write a Python Program to use the scikit-learn package in python to implement the regression model and its related methods.	CO2
9	Use publicly available medical dataset and perform univariate and bivariate analysis and compare results of above analysis for two datasets.	CO2
10	Case study to scale and improve BI systems	CO3
	Content Beyond Sullabus:	CO4
11	CBS 1: Correlation Analysis	
12	CBS 2: Machine Learning Techniques for Data Science	CO4

ASSIGNMENT NO. 1

TITLE

Python Program to Retrieve File Mode Information and Display Local & GMT Time Components.

PROBLEM STATEMET

Python program to get information about the file pertaining to the file mode and to get time values with components using local time and gm time.

OBJECTIVE

- To develop a Python program that retrieves and displays information about a given file, including its mode (e.g., read, write, execute permissions).
- To extract and present the current date and time components using the local time of the system.
- To extract and present the current date and time components in GMT (Greenwich Mean Time).
- To demonstrate the use of Python's built-in modules such as os, time, or similar libraries for file and time operations.
- To help users understand how file attributes and system time information can be accessed and formatted in Python.

PLATFORM REQUIRED

- **Operating System:** Windows or Linux
- **Software/Tools:** Anaconda, Spyder, Jypter Notebook, Google Colab, Python 3.x

Text Editor (such as Visual Studio Code, Sublime Text, or any IDE)

THEORY

Working with File Mode and Time Functions in Python

we aim to develop a Python program that performs two key functions:

1. Retrieving file-related information, particularly the file mode.
2. Extracting and displaying time-related data, including components of local time and Greenwich Mean Time (GMT).

1. File Mode Information

Every file in an operating system has associated metadata, such as:

- File name
- File size
- File permissions (read, write, execute)
- File creation and modification times
- File mode (shows how the file was opened — e.g., read 'r', write 'w', append 'a')

In Python, the os and stat modules are commonly used to extract such information. For instance:

- os.stat() returns an object containing the file's status, including its mode.
- stat.filemode() can convert this mode into a human-readable string.
- Convert the mode into a readable format using oct() function.

For Example,

```
import os

import stat

print(stat.filemode(os.stat("example.txt").st_mode))
```

2. Time Values and Time Components

Python's time module allows us to work with time in various formats:

- Local time: Refers to the time based on the computer's timezone.
- GMT/UTC time: Refers to Coordinated Universal Time, used as a standard reference.

Two useful functions:

- time.localtime() – returns the local time in a structured format.
- time.gmtime() – returns the GMT/UTC time in a similar structure.
- Use time.strftime() to format the time components (year, month, day, hour, minute, second).

Each of these returns a struct_time object with the following components:

- tm_year, tm_mon, tm_mday (date components)
- tm_hour, tm_min, tm_sec (time components)
- tm_wday, tm_yday, tm_isdst (weekday, day of year, daylight saving flag)

Example:

```
import time

lt = time.localtime()

gt = time.gmtime()

print("Local time:", lt)

print("GMT:", gt)
```

QUESTIONS

1. What is the purpose of using os and time modules in Python for file handling and time operations?
2. Explain the difference between local time and GMT (Greenwich Mean Time) in Python.
3. What is file mode in Python and how can it be retrieved programmatically?
4. How does the os.stat() function help in accessing file metadata? List the important attributes it returns.
5. Differentiate between time.localtime() and time.gmtime() with suitable examples.
6. Why is it important to extract time values in both local and GMT formats during file processing?
7. Describe the structure of the time.struct_time object returned by local and GMT time functions.

CONCLUSION

This program demonstrates how Python can be used to retrieve file mode details and extract time components using both local time and GMT, enabling effective file handling and time-related operations.

ASSIGNMENT NO. 2

TITLE

Performing Array Operations Using NumPy in Python.

PROBLEM STATEMET

Perform Array operations using Numpy: Write a Python program to

a. Create N-dim arrays
b. Use inbuilt function (zeros, ones, full, eye) to create array
c. Use shape, reshape, transpose function
d. Use sort function and Sorting along axes
e. Use arange, linspace, logspace function
f. Array Indexing, slicing, Joining, Splitting, and Searching arrays

OBJECTIVE

- To understand the concept and creation of N-dimensional arrays using NumPy.
- To explore built-in NumPy functions like zeros(), ones(), full(), and eye() for array initialization.
- To manipulate array dimensions using functions such as shape, reshape, and transpose.
- To apply sorting operations using sort() function and perform sorting along specified axes.

- To generate sequences using NumPy functions like arange(), linspace(), and logspace().
- To perform essential array operations such as indexing, slicing, joining, splitting, and searching.

PLATFORM REQUIRED

- **Operating System:** Windows or Linux
- **Software/Tools:** Anaconda, Spyder, Jypter Notebook, Google Colab, Python 3.x

Text Editor (such as Visual Studio Code, Sublime Text, or any IDE)

THEORY

Array Operations Using NumPy in Python

NumPy (Numerical Python) is a powerful library in Python that provides support for large, multi-dimensional arrays and matrices, along with a rich collection of mathematical functions to operate on these arrays efficiently. It is widely used in data science, machine learning, scientific computing, and engineering applications.

a. Creating N-dimensional Arrays

- NumPy arrays can be created using the numpy.array() function.
- N-dimensional arrays (ndarrays) allow efficient storage and manipulation of numerical data.

Example:

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
```

b. **Using Inbuilt Functions to Create Arrays**

NumPy provides built-in functions to create arrays with specific values:

- zeros(shape) → Creates an array filled with zeros.
- ones(shape) → Creates an array filled with ones.
- full(shape, fill_value) → Creates an array with a constant value.
- eye(n) → Creates an identity matrix.

Example:

```
np.zeros((2, 3))
np.ones((3, 3))
np.full((2, 2), 5)
```

```
np.eye(3)
```

c. **Using shape, reshape, and transpose Functions**

- shape → Returns the dimensions (rows, columns) of the array.
- reshape → Changes the shape of an existing array without changing data.
- transpose → Swaps rows and columns.

Example:

```
arr = np.array([[1, 2], [3, 4], [5, 6]])
arr.shape       # Output: (3, 2)
arr.reshape(2, 3)
arr.T           # Transpose the array
```

d. **Sorting Arrays and Sorting Along Axes**

- np.sort() → Sorts an array.
- Sorting along axes allows sorting of rows or columns individually.

Example:

```
arr = np.array([[3, 1, 2], [6, 4, 5]])
np.sort(arr, axis=0)  # Sort columns
np.sort(arr, axis=1)  # Sort rows
```

e. **Using arange, linspace, and logspace Functions**

- arange(start, stop, step) → Returns evenly spaced values within a range.
- linspace(start, stop, num) → Returns evenly spaced numbers over a specified interval.
- logspace(start, stop, num) → Returns numbers spaced evenly on a log scale.

Example:

```
np.arange(1, 10, 2)      # [1, 3, 5, 7, 9]
np.linspace(0, 1, 5)     # [0. , 0.25, 0.5, 0.75, 1. ]
np.logspace(1, 3, 3)     # [10, 100, 1000]
```

f. **Array Indexing, Slicing, Joining, Splitting, Searching**

- Indexing: Access individual elements using index.
- Slicing: Access subsets of arrays.
- Joining: Combine arrays using functions like concatenate(), hstack(), vstack().

- Splitting: Divide arrays using split(), hsplit(), vsplit().
- Searching: Use where() or searchsorted() to find element positions.

Example:

```
arr = np.array([10, 20, 30, 40])
arr[1]              # Indexing → 20
arr[1:3]            # Slicing → [20 30]
np.where(arr > 20)  # Searching → (array([2, 3]),)
```

QUESTIONS

1. Explain the concept of N-dimensional arrays in NumPy. How are they created?
2. Differentiate between `zeros()`, `ones()`, `full()`, and `eye()` functions in NumPy.
3. Explain the use of `shape`, `reshape`, and `transpose` functions with examples.
4. How does NumPy handle sorting of arrays? Explain sorting along different axes.
5. Describe the difference between `arange()`, `linspace()`, and `logspace()` with examples.
6. Describe various techniques to search values in NumPy arrays.

CONCLUSION

By performing array operations using NumPy, we gain powerful tools for handling and manipulating large datasets efficiently. Through this program, we explored the creation of N-dimensional arrays, used built-in functions like zeros(), ones(), full(), and eye(), and applied important functions such as reshape(), transpose(), and sorting techniques. Additionally, we practiced advanced operations like indexing, slicing, joining, splitting, and searching arrays. These foundational skills are essential for data analysis, scientific computing, and machine learning applications in Python.

ASSIGNMENT NO. 3

TITLE

Exploring DataFrame Operations for Data Analysis with Pandas.

PROBLEM STATEMET

Perform Data Manipulation operations using Pandas package on dataframes.

OBJECTIVE

- Understand the basics of Pandas and DataFrame structure.
- Perform data selection, filtering, and sorting operations.
- Handle missing data using Pandas functions.
- Apply operations like adding, deleting, and renaming columns.
- Use groupby, merging, and joining for data aggregation.

PLATFORM REQUIRED

- **Operating System:** Windows or Linux
- **Software/Tools:** Anaconda, Spyder, Jypter Notebook, Google Colab, Python 3.x

Text Editor (such as Visual Studio Code, Sublime Text, or any IDE)

THEORY

1. **Importing Pandas Library:**
   - Install Pandas using `pip install pandas` if not already installed.
   - Import the Pandas library at the beginning of the script.

     import pandas as pd

2. **Creating and Loading DataFrames:**

- Create a DataFrame manually or load data from a CSV file.

```
# Creating a DataFrame manually

data = {

    'Name': ['John', 'Anna', 'Peter', 'Linda'],

    'Age': [28, 24, 35, 32],

    'City': ['New York', 'Paris', 'Berlin', 'London'],

    'Salary': [70000, 65000, 80000, 90000]

}
```

```
df = pd.DataFrame(data)
print("DataFrame created manually:\n", df)
# Loading data from a CSV file
# df = pd.read_csv('path_to_your_file.csv')
```

3. **Inspecting Data in a DataFrame:**

- View the first few rows, the structure, and summary statistics of the DataFrame.

```
# Viewing the first few rows of the DataFrame
print("\nFirst few rows of the DataFrame:\n", df.head())
# Viewing the structure of the DataFrame
print("\nInformation about the DataFrame:\n")
df.info()
# Summary statistics of the DataFrame
print("\nSummary statistics of the DataFrame:\n", df.describe())
```

4. **Selecting and Filtering Data:**

- Select specific columns, rows, and filter data based on conditions.

```
# Selecting specific columns
selected_columns = df[['Name', 'Salary']]
print("\nSelected columns (Name and Salary):\n", selected_columns)
# Filtering data based on a condition
high_salary = df[df['Salary'] > 70000]
print("\nRows where Salary is greater than 70000:\n", high_salary)
```

5. **Adding, Modifying, and Deleting Columns:**

- Add new columns, modify existing ones, and delete columns.

```
# Adding a new column
```

```
df['Experience'] = [5, 2, 8, 7]
print("\nDataFrame after adding 'Experience' column:\n", df)
# Modifying an existing column
df['Salary'] = df['Salary'] + 5000
print("\nDataFrame after modifying 'Salary' column:\n", df)
# Deleting a column
df = df.drop('City', axis=1)
print("\nDataFrame after deleting 'City' column:\n", df)
```

6. **Sorting Data:**

- Sort the DataFrame by a specific column.

```
# Sorting the DataFrame by 'Age'
sorted_df = df.sort_values(by='Age')
print("\nDataFrame sorted by Age:\n", sorted_df)
```

7. **Grouping Data and Aggregation:**

- Group the data by a specific column and apply aggregation functions.

```
# Grouping by 'Experience' and calculating mean salary
grouped_df = df.groupby('Experience')['Salary'].mean()
print("\nMean Salary grouped by Experience:\n", grouped_df)
```

8. **Merging and Joining DataFrames:**

- Combine two DataFrames using merge and join operations.

```
# Example DataFrame to join with
additional_data = {
    'Name': ['John', 'Anna', 'Peter', 'Linda'],
    'Department': ['IT', 'HR', 'Finance', 'Marketing']
```

```
}
df2 = pd.DataFrame(additional_data)
# Merging the DataFrames on 'Name'
merged_df = pd.merge(df, df2, on='Name')
print("\nMerged DataFrame:\n", merged_df)
```

9. **Handling Missing Data:**

- Deal with missing data by filling or dropping them.

```
# Example DataFrame with missing values
df_with_nan = df.copy()
df_with_nan.loc[2, 'Age'] = None
# Filling missing values
df_filled = df_with_nan.fillna(30)
print("\nDataFrame with missing values filled:\n", df_filled)
# Dropping rows with missing values
df_dropped = df_with_nan.dropna()
print("\nDataFrame with missing values dropped:\n", df_dropped)
```

QUESTIONS

1. Explain the significance of data manipulation in data analysis.
2. What are the common operations used to inspect a DataFrame?
3. How can you filter data based on conditions in Pandas?
4. Discuss different ways to handle missing values in a DataFrame.
5. What is the use of the `groupby()` function? Provide an example.
6. Compare `loc[]` and `iloc[]` with suitable examples.
7. How do you sort a DataFrame based on multiple columns?
8. Explain the difference between `merge()`, `concat()`, and `join()` operations in Pandas.

CONCLUSION

This lab provides a comprehensive overview of data manipulation using the Pandas library, highlighting essential techniques for analyzing and transforming data in DataFrames.

ASSIGNMENT NO. 4

TITLE

Apply and explore multiple types of plots using Matplotlib package.

PROBLEM STATEMET

To write a Python program that demonstrates how to apply and explore multiple types of plots using the Matplotlib package, including line plots, bar plots, histograms, scatter plots, pie charts, and more.

OBJECTIVE

- To understand the importance of data visualization in data analysis.
- To learn how to use the Matplotlib library for creating various types of plots.
- To apply and customize different plot types such as:Line plot, Bar chart,Histogram,Scatter plot,Pie chart,Subplots
- To practice setting plot properties such as title, axis labels, legends, gridlines, and colors.
- To analyze and interpret data visually using appropriate plotting techniques.

PLATFORM REQUIRED

- **Operating System:** Windows or Linux
- **Software/Tools:** Anaconda, Spyder, Jypter Notebook, Google Colab, Python 3.x

  Text Editor (such as Visual Studio Code, Sublime Text, or any IDE)

THEORY

1. **Importing Matplotlib Library:**
   - Install Matplotlib using `pip install matplotlib` if not already installed.
   - Import the Matplotlib library at the beginning of the script.

     import matplotlib.pyplot as plt

2. **Creating a Line Plot:**

- Create a simple line plot to visualize trends over time or sequences.

```
# Data for plotting
x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 7, 11]
# Creating the line plot
plt.figure()
plt.plot(x, y, marker='o')
plt.title('Line Plot')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.grid(True)
plt.show()
```

3. **Creating a Bar Plot:**

- Use a bar plot to compare quantities across different categories.

```
# Data for plotting
categories = ['A', 'B', 'C', 'D']
values = [5, 7, 3, 8]
# Creating the bar plot
plt.figure()
plt.bar(categories, values, color='skyblue')
plt.title('Bar Plot')
plt.xlabel('Categories')
plt.ylabel('Values')
```

```
plt.show()
```

4. **Creating a Histogram:**

- Create a histogram to display the distribution of a dataset.

```
# Data for plotting
data = [1, 2, 2, 2, 3, 3, 3, 3, 4, 4, 5, 5, 6, 7, 8]
# Creating the histogram
plt.figure()
plt.hist(data, bins=5, color='green', edgecolor='black')
plt.title('Histogram')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.show()
```

5. **Creating a Scatter Plot:**

- Create a scatter plot to visualize relationships between two variables.

```
# Data for plotting
x = [5, 7, 8, 7, 2, 17, 2, 9, 4, 11]
y = [99, 86, 87, 88, 100, 86, 103, 87, 94, 78]
# Creating the scatter plot
plt.figure()
plt.scatter(x, y, color='red')
plt.title('Scatter Plot')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.show()
```

6. **Creating a Pie Chart:**

- Use a pie chart to show the proportions of different categories in a dataset.

```
# Data for plotting
labels = ['Category A', 'Category B', 'Category C', 'Category D']
sizes = [15, 30, 45, 10]
colors = ['gold', 'yellowgreen', 'lightcoral', 'lightskyblue']
# Creating the pie chart
plt.figure()
plt.pie(sizes, labels=labels, colors=colors, autopct='%1.1f%%', startangle=140)
plt.title('Pie Chart')
plt.show()
```

7. **Creating a Box Plot:**

- Create a box plot to visualize the spread and skewness of a dataset.

```
# Data for plotting
data = [np.random.normal(0, std, 100) for std in range(1, 4)]
# Creating the box plot
plt.figure()
plt.boxplot(data, vert=True, patch_artist=True)
plt.title('Box Plot')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.show()
```

8. **Creating a Heatmap:**

- Create a heatmap to visualize data as a color-encoded matrix.

```
# Data for plotting
data = np.random.rand(10, 10)
# Creating the heatmap
plt.figure()
plt.imshow(data, cmap='viridis', interpolation='none')
plt.colorbar()
plt.title('Heatmap')
plt.show()
```

QUESTIONS

1. Explain the difference between a bar chart and a histogram with examples.
2. How can you customize the plot title, axis labels, and grid in a Matplotlib chart?
3. What are the use cases of a pie chart in data analysis?
4. How can you add legends to a plot in Matplotlib?
5. Describe how scatter plots are useful for showing relationships in data.
6. Write a Python script to create a histogram of 100 random numbers generated using NumPy.
7. How do you change the color, line style, and markers in Matplotlib plots?
8. Compare the use of `plt.bar()` and `plt.barh()` with examples.
9. What is the purpose of `plt.subplot()`? How is it different from `plt.subplots()`?
10. How can you display multiple plots in a single figure using Matplotlib?

CONCLUSION

This program provides an overview of various plotting techniques using Matplotlib, demonstrating how to visualize data through different types of plots.

ASSIGNMENT NO. 5

TITLE

Statistical and Comparison Operations on CSV Data using Python.

PROBLEM STATEMET

Write a Python Program to import a CSV file and perform various Statistical and Comparison operations on rows/columns.

OBJECTIVE

- To understand how to import and read data from a CSV file using Python.
- To perform statistical operations such as mean, median, mode, standard deviation, etc., on rows and columns.
- To apply comparison and conditional operations on dataset values.
- To explore data using pandas for effective data manipulation and analysis.
- To gain practical experience in working with real-world datasets using Python and pandas.

PLATFORM REQUIRED

- **Operating System:** Windows or Linux
- **Software/Tools:** Anaconda, Spyder, Jypter Notebook, Google Colab, Python 3.x

  Text Editor (such as Visual Studio Code, Sublime Text, or any IDE)

THEORY

1. **Importing Required Libraries:**
   - Install the Pandas library using `pip install pandas` if not already installed.
   - Import the Pandas library for handling CSV files and performing data operations.

     import pandas as pd

2. **Loading the CSV File:**

- Use the read_csv function from Pandas to load the CSV file into a DataFrame.

  # Load the CSV file

  df = pd.read_csv('yourfile.csv')

  # Display the first few rows of the DataFrame

  print(df.head())

3. **Performing Statistical Operations:**

- **Descriptive Statistics:** Use describe() to get a summary of the numerical columns.

  # Descriptive statistics

  print(df.describe())

  Mean, Median, Mode:

  # Mean of each column

  print(df.mean())

  # Median of each column

```
print(df.median())

# Mode of each column

print(df.mode())
```

Sum, Min, Max:

```
# Sum of each column

print(df.sum())

# Minimum value of each column

print(df.min())

# Maximum value of each column

print(df.max())
```

Standard Deviation, Variance:

```
# Standard deviation of each column

print(df.std())

# Variance of each column

print(df.var())
```

4. **Performing Comparison Operations:**

- **Filtering Rows Based on a Condition:**

```
# Filter rows where a specific column's value is greater than a threshold

filtered_df = df[df['column_name'] > threshold_value]

print(filtered_df)
```

- **Comparing Columns:**

```
# Compare values between two columns

comparison = df['column1'] > df['column2']

print(comparison)
```

- **Group By Operations:**

```
# Group by a column and calculate the mean of each group
grouped_df = df.groupby('column_name').mean()
print(grouped_df)
```

5. **Applying Custom Functions:**

- **Applying a Function to a Column:**

```
# Define a custom function
def custom_function(x):
    return x * 2
# Apply the function to a column
df['new_column'] = df['column_name'].apply(custom_function)
print(df)
```

6. **Exporting the Results:**

- **Save the DataFrame to a New CSV File:**

```
# Export the DataFrame to a new CSV file
df.to_csv('output.csv', index=False)
```

QUESTIONS

1. Which functions are used to calculate statistical measures like mean, median, and standard deviation in pandas?
2. How do you perform column-wise and row-wise operations in a DataFrame?
3. Explain how to apply conditional filtering and comparison between DataFrame columns.
4. How can you compare values of two columns and generate a new result column?
5. How do missing or null values affect statistical calculations, and how can they be handled?
6. Write a Python code snippet to import a CSV file and display only the rows where the values in a specific column exceed the average of that column.

CONCLUSION

This program demonstrates how to efficiently use Python and the pandas library to import CSV data and perform essential statistical and comparison operations. It enables better understanding, analysis, and decision-making by extracting meaningful insights from structured data.

ASSIGNMENT NO. 6

TITLE

Comprehensive Data Exploration Using Pandas in Python.

PROBLEM STATEMET

Write a Python Program to import any CSV file to Pandas DataFrame and perform the following:

i. Visualize the first and last 10 records

ii. Get the shape, index and column details

iii. Select/Delete the records(rows)/columns based on conditions.

iv. Perform ranking and sorting operations.

v. Do require statistical operations on the given columns.

vi. Find the count and uniqueness of the given categorical values.

vii. Rename single/multiple columns.

OBJECTIVE

- Manipulate Data Based on Conditions.
- Execute Sorting and Ranking Operations
- Conduct Descriptive Statistical Analysis
- Analyze Categorical Data

PLATFORM REQUIRED

- **Operating System:** Windows or Linux
- **Software/Tools:** Anaconda, Spyder, Jypter Notebook, Google Colab, Python 3.x

Text Editor (such as Visual Studio Code, Sublime Text, or any IDE)

THEORY

1. **Importing Required Libraries:**
   - Install the Pandas library using `pip install pandas` if not already installed.
   - Import the Pandas library for handling CSV files and performing data operations.

```
import pandas as pd
```

2. **Loading the CSV File:**

- Use the read_csv function from Pandas to load the CSV file into a DataFrame.

```
# Load the CSV file

df = pd.read_csv('yourfile.csv')

# Display the first few rows of the DataFrame

print(df.head())
```

3. **Visualize the First and Last 10 Records:**

- Use head() and tail() functions to display the first and last 10 records of the DataFrame.

```
# Display the first 10 records

print(df.head(10))

# Display the last 10 records

print(df.tail(10))
```

4. **Get the Shape, Index, and Column Details:**

- Use shape, index, and columns attributes to get the size, index, and column details of the DataFrame.

```
# Get the shape of the DataFrame

print("Shape of DataFrame:", df.shape)

# Get the index of the DataFrame

print("Index of DataFrame:", df.index)

# Get the column names of the DataFrame

print("Columns in DataFrame:", df.columns)
```

5. **Select/Delete Records (Rows)/Columns Based on Conditions:**

- **Select Rows/Columns:**

```
# Select specific columns

selected_columns = df[['column1', 'column2']]

print(selected_columns)

# Select rows based on a condition

selected_rows = df[df['column_name'] > threshold_value]

print(selected_rows)
```

- **Delete Rows/Columns:**

```
# Delete a column
```

```
df = df.drop('column_name', axis=1)

print(df)


# Delete rows based on a condition

df = df[df['column_name'] <= threshold_value]

print(df)
```

6. Perform Ranking and Sorting Operations:

- **Ranking:**

```
# Rank the values in a specific column

df['ranked_column'] = df['column_name'].rank()

print(df)
```

- **Sorting:**

```
# Sort by a specific column in ascending order

sorted_df = df.sort_values(by='column_name')

print(sorted_df)


# Sort by multiple columns

sorted_df = df.sort_values(by=['column1', 'column2'], ascending=[True, False])

print(sorted_df)
```

7. **Perform Required Statistical Operations on Given Columns:**

- **Calculate mean, median, sum, etc., for specific columns:**

```
# Calculate the mean of a column

print("Mean of column:", df['column_name'].mean())

# Calculate the sum of a column
```

```
print("Sum of column:", df['column_name'].sum())

# Calculate the median of a column

print("Median of column:", df['column_name'].median())
```

8. **Find the Count and Uniqueness of Categorical Values:**

- **Count occurrences of unique values:**

```
# Count the number of occurrences of each unique value in a column

print(df['categorical_column'].value_counts())

# Get the unique values in a column

print(df['categorical_column'].unique())
```

9. **Rename Single/Multiple Columns:**

- **Rename columns:**

```
# Rename a single column

df.rename(columns={'old_column_name': 'new_column_name'}, inplace=True)

print(df.columns)

# Rename multiple columns

df.rename(columns={'old_column1': 'new_column1', 'old_column2': 'new_column2'},
inplace=True)

print(df.columns)
```

QUESTIONS

1. How can you retrieve the shape, index range, and column names of a DataFrame named df? Write the necessary commands.
2. Write code to select rows where the Salary column is greater than 50,000 and delete the Address column from the DataFrame.
3. Explain how to rank students based on their Marks and sort the DataFrame in descending order of Total. Provide the code.
4. Write Python code to find the number of unique departments in a Department column and rename the column EmpName to Employee_Name.

CONCLUSION

This lab exercise demonstrates the use of the Pandas library in Python for importing CSV data and carrying out a variety of operations, including data selection, deletion, ranking, sorting, and statistical analysis.

ASSIGNMENT NO. 7

TITLE

End-to-End Data Wrangling and Plotting Using Pandas Series and DataFrames.

.**PROBLEM STATEMET**

Write a Python Program to import any CSV file to Pandas DataFrame and perform the following:

i. Handle missing data by detecting and dropping/ filling missing values.

ii. Transform data using different methods.

iii. Detect and filter outliers.

iv. Perform Vectorized String operations on Pandas Series.

v. Visualize data using Line Plots, Bar Plots, Histograms, Density Plots and Scatter Plots..

OBJECTIVE

- import CSV data into a Pandas DataFrame for analysis.
- Identify and handle missing values by dropping or filling them.
- Apply various data transformation techniques to clean and prepare data.
- Detect and filter outliers to improve data quality.
- Perform efficient vectorized string operations on textual data.
- Create visualizations using line plots, bar plots, histograms, density plots, and scatter plots.

PLATFORM REQUIRED

- **Operating System:** Windows or Linux
- **Software/Tools:** Anaconda, Spyder, Jypter Notebook, Google Colab, Python 3.x

Text Editor (such as Visual Studio Code, Sublime Text, or any IDE)

THEORY

1. **Importing Required Libraries:**
   - Install the required libraries using `pip install pandas matplotlib seaborn` if not already installed.
   - Import the necessary libraries for data handling and visualization.

```
import pandas as pd

import numpy as np
```

```
import matplotlib.pyplot as plt

import seaborn as sns
```

2. **Loading the CSV File:**

- Use the read_csv function from Pandas to load the CSV file into a DataFrame.

```
# Load the CSV file

df = pd.read_csv('yourfile.csv')

# Display the first few rows of the DataFrame

print(df.head())
```

3. **Handling Missing Data:**

- **Detect Missing Data:**

```
# Check for missing values

print(df.isnull().sum())
```

- **Drop Missing Data:**

```
# Drop rows with missing values

df_cleaned = df.dropna()

print(df_cleaned.head())
```

- **Fill Missing Data:**

```
# Fill missing values with the mean of the column

df_filled = df.fillna(df.mean())

print(df_filled.head())
```

4. **Transforming Data:**

- **Applying Log Transformation:**

```
# Apply log transformation to a specific column

df['log_transformed_column'] = np.log(df['column_name'])
```

```
print(df.head())
```

- **Normalizing Data:**

```
# Normalize data using Min-Max scaling

df['normalized_column'] = (df['column_name'] - df['column_name'].min()) / (df['column_name'].max() - df['column_name'].min())

print(df.head())
```

- **Standardizing Data:**

```
# Standardize data to have a mean of 0 and standard deviation of 1

df['standardized_column'] = (df['column_name'] - df['column_name'].mean()) / df['column_name'].std()

print(df.head())
```

5. **Detecting and Filtering Outliers:**

- **Using Z-Score to Detect Outliers:**

```
# Calculate Z-scores for a column

df['z_score'] = (df['column_name'] - df['column_name'].mean()) / df['column_name'].std()

# Filter out outliers where the Z-score is greater than 3 or less than -3

df_no_outliers = df[(df['z_score'] > -3) & (df['z_score'] < 3)]

print(df_no_outliers.head())
```

- **Using IQR (Interquartile Range) Method:**

```
# Calculate Q1 and Q3

Q1 = df['column_name'].quantile(0.25)

Q3 = df['column_name'].quantile(0.75)

IQR = Q3 - Q1



# Filter out outliers
```

```
df_no_outliers = df[~((df['column_name'] < (Q1 - 1.5 * IQR)) | (df['column_name'] > (Q3 + 1.5 *
    IQR)))]

print(df_no_outliers.head())
```

6. **Performing Vectorized String Operations on Pandas Series:**

- **String Operations:**

```
# Convert a column to lowercase

df['lowercase_column'] = df['string_column'].str.lower()

print(df.head())

# Replace specific values in a string column

df['replaced_column'] = df['string_column'].str.replace('old_value', 'new_value')

print(df.head())

# Check if strings contain a specific substring

df['contains_substring'] = df['string_column'].str.contains('substring')

print(df.head())
```

7. **Visualizing Data Using Different Plot Types:**

- **Line Plot:**

```
# Plot a line graph

plt.figure(figsize=(10, 6))

plt.plot(df['column1'], df['column2'])

plt.title('Line Plot')

plt.xlabel('X-axis Label')

plt.ylabel('Y-axis Label')

plt.show()
```

- **Bar Plot:**

```
# Plot a bar graph

plt.figure(figsize=(10, 6))

df['categorical_column'].value_counts().plot(kind='bar')

plt.title('Bar Plot')

plt.xlabel('Categories')

plt.ylabel('Frequency')

plt.show()
```

- **Histogram:**

```
# Plot a histogram

plt.figure(figsize=(10, 6))

df['numeric_column'].plot(kind='hist', bins=20)

plt.title('Histogram')

plt.xlabel('Value')

plt.ylabel('Frequency')

plt.show()
```

- **Density Plot:**

```
# Plot a density plot

plt.figure(figsize=(10, 6))

df['numeric_column'].plot(kind='density')

plt.title('Density Plot')

plt.xlabel('Value')

plt.show()
```

- **Scatter Plot:**

```
# Plot a scatter plot
```

```
plt.figure(figsize=(10, 6))

plt.scatter(df['column1'], df['column2'])

plt.title('Scatter Plot')

plt.xlabel('X-axis Label')

plt.ylabel('Y-axis Label')

plt.show()
```

QUESTIONS

1. Explain two different techniques to handle missing data in Pandas. Demonstrate using dropna() and fillna() functions.

2. What are data transformation techniques? Write Python code to apply normalization or standardization on a numerical column.

3. How can you detect outliers in a dataset using statistical methods (e.g., IQR or Z-score)? Write code to remove outliers from a column.

4. List and explain any three vectorized string functions available in Pandas. Provide examples using a sample Pandas Series..

CONCLUSION

This lab exercise demonstrates the use of the Pandas library in Python for advanced data manipulation and analysis tasks, including handling missing data, transforming data, detecting outliers, and visualizing data using different plot types..

ASSIGNMENT NO. 8

TITLE

Implementation of Regression Models using Scikit-learn in Python.

PROBLEM STATEMET

Write a Python Program to use the scikit-learn package in python to implement the regression model and its related methods.

OBJECTIVE

- To understand the concept and importance of regression analysis in data science.
- To evaluate model performance using metrics such as Mean Squared Error (MSE), R^2 score, etc.
- To apply methods like model fitting, prediction, and visualization of regression results.
- To develop hands-on skills in feature selection, data preprocessing, and model validation.

PLATFORM REQUIRED

- **Operating System:** Windows or Linux
- **Software/Tools:** Anaconda, Spyder, Jypter Notebook, Google Colab, Python 3.x

Text Editor (such as Visual Studio Code, Sublime Text, or any IDE)

THEORY

1. **Importing Required Libraries:**
   - Install the required libraries using `pip install scikit-learn pandas numpy matplotlib seaborn` if not already installed.
   - Import the necessary libraries for data handling, model creation, and visualization.

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error, r2_score
```

2. **Loading the Dataset:**

- Load the dataset into a Pandas DataFrame. Ensure that the dataset contains continuous numeric data suitable for regression analysis.

```
# Load the dataset

df = pd.read_csv('your_dataset.csv')

# Display the first few rows of the DataFrame

print(df.head())
```

3. **Data Preprocessing:**

- **Handling Missing Data:**

```
# Check for missing values

print(df.isnull().sum())

# Fill or drop missing values

df.fillna(df.mean(), inplace=True)
```

- **Feature Selection**

```
# Select the independent and dependent variables (features and target)

X = df[['feature1', 'feature2', 'feature3']]  # Independent variables

y = df['target']  # Dependent variable
```

- **Splitting the Dataset:**

```
# Split the dataset into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

4. **Implementing the Regression Model:**

- **Creating the Linear Regression Model:**

```
# Initialize the Linear Regression model

model = LinearRegression()

# Train the model using the training data

model.fit(X_train, y_train)
```

- **Making Predictions:**

```
# Make predictions on the testing data

y_pred = model.predict(X_test)
```

5. **Evaluating the Model:**

- **Calculating Mean Squared Error (MSE):**

```
# Calculate Mean Squared Error

mse = mean_squared_error(y_test, y_pred)

print(f'Mean Squared Error: {mse}')
```

- **Calculating R-squared Value:**

```
# Calculate R-squared value

r2 = r2_score(y_test, y_pred)

print(f'R-squared Value: {r2}')
```

- **Visualizing the Regression Line:**

```
# Plot the actual vs predicted values
```

```
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred, color='blue')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color='red', linewidth=2)
plt.xlabel('Actual')
plt.ylabel('Predicted')
plt.title('Actual vs Predicted')
plt.show()
```

6. **Exploring Additional Methods:**

- **Coefficients and Intercept:**

```
# Print the coefficients and intercept
print(f'Coefficients: {model.coef_}')
print(f'Intercept: {model.intercept_}')
```

- **Cross-Validation:**

```
from sklearn.model_selection import cross_val_score
# Perform cross-validation
cv_scores = cross_val_score(model, X, y, cv=5)
print(f'Cross-Validation Scores: {cv_scores}')
print(f'Average Cross-Validation Score: {np.mean(cv_scores)}')
```

QUESTIONS

1. What is regression analysis and why is it important in data science?
2. Explain the difference between linear and multiple regression models.
3. What is the role of the `scikit-learn` library in building regression models?
4. Define and explain the significance of evaluation metrics such as MSE, RMSE, and R^2 score.
5. How does feature scaling affect regression performance?

CONCLUSION

This lab exercise demonstrates the implementation of a linear regression model using the scikit-learn library in Python. The steps include loading and preprocessing data, training the model, making predictions, and evaluating the model's performance with different metrics and visualizations.

ASSIGNMENT NO. 9

TITLE

Comparative Analysis of Medical Datasets Using Univariate and Bivariate Statistical Techniques.

PROBLEM STATEMET

Use publicly available medical dataset and perform univariate and bivariate analysis and compare results of above analysis for two datasets.

OBJECTIVE

- Perform univariate and bivariate analysis on medical datasets.

- Identify data patterns and relationships between variables.
- Compare analysis results across two different datasets.
- Enhance data handling and visualization skills using Python.
- Apply statistical methods to real-world healthcare data.

PLATFORM REQUIRED

- **Operating System:** Windows or Linux
- **Software/Tools:** Anaconda, Spyder, Jypter Notebook, Google Colab, Python 3.x

Text Editor (such as Visual Studio Code, Sublime Text, or any IDE)

THEORY

Univariate and bivariate analyses are fundamental statistical techniques used to understand the distribution and relationship of data variables. In the context of medical datasets, these methods help uncover patterns, trends, and associations that are crucial for diagnosis, treatment planning, and healthcare decision-making.

- Univariate Analysis focuses on analyzing a single variable at a time. It involves summarizing data through measures such as mean, median, mode, variance, and visualizations like histograms, box plots, and bar charts. For instance, analyzing patient age distribution or blood pressure levels gives insights into the central tendency and spread of those individual variables.
- Bivariate Analysis examines the relationship between two variables. It helps identify how one variable influences or correlates with another. Techniques such as scatter plots, correlation coefficients, and cross-tabulations are used. For example, investigating the relationship between BMI and diabetes risk or age and heart disease helps in understanding predictive patterns.

By applying these analyses to two different publicly available medical datasets (e.g., one on heart disease and another on diabetes), students can compare how variable relationships vary across datasets. This comparison enhances analytical thinking and provides a deeper understanding of medical data characteristics, helping in real-world data science applications within healthcare.

Datasets:

Choose two publicly available medical datasets, such as:

- **Dataset 1:** Example: Diabetes dataset from the UCI Machine Learning Repository
- **Dataset 2:** Example: Heart Disease dataset from the UCI Machine Learning Repository

Experiment Steps:

Part 1: Data Collection and Preparation

1. **Download the Datasets:**
   - Download the selected medical datasets and load them into your analysis tool (Python/R).
2. **Data Cleaning:**
   - Inspect the datasets for missing values, outliers, and inconsistent data.
   - Handle missing values appropriately (e.g., imputation or removal).
   - Convert categorical variables to numerical form if necessary.
3. **Data Exploration:**
   - Understand the structure of the data by checking the number of rows and columns, data types, and basic statistics (mean, median, mode, etc.).

Part 2: Univariate Analysis

1. **Univariate Analysis on Dataset 1:**
   - Select key variables and perform univariate analysis.
   - Generate descriptive statistics (mean, median, mode, standard deviation).
   - Create visualizations (histograms, box plots) for continuous variables.
   - Create frequency tables and bar charts for categorical variables.
2. **Univariate Analysis on Dataset 2:**
   - Repeat the same analysis steps as Dataset 1.

Part 3: Bivariate Analysis

1. **Bivariate Analysis on Dataset 1:**
   - Select pairs of variables to study relationships.
   - Perform bivariate analysis by calculating correlation coefficients for continuous variables.
   - Generate scatter plots, heatmaps, or other relevant plots to visualize relationships.
   - Analyze the relationship between categorical variables using cross-tabulation and chi-square tests.
2. **Bivariate Analysis on Dataset 2:**
   - Repeat the same analysis steps as Dataset 1.

Part 4: Comparison and Interpretation

1. **Compare Univariate Analysis Results:**
   - Compare the distributions, central tendencies, and variances between the two datasets.
   - Discuss the key similarities and differences in the univariate analysis results.
2. **Compare Bivariate Analysis Results:**
   - Compare the relationships between variables in the two datasets.
   - Discuss the key similarities and differences in the bivariate analysis results.
3. **Interpretation:**
   - Summarize the findings from the analyses.
   - Discuss how the results can be used in a real-world medical context.

QUESTIONS

1. What are the key differences between univariate and bivariate analysis?
2. What types of plots did you use for univariate analysis, and what do they reveal about the data?.
3. Which variable pairs were analyzed in bivariate analysis?
4. What correlations did you find between variables such as age vs. disease presence, or BMI vs. glucose level?

CONCLUSION

The analysis of two medical datasets using univariate and bivariate techniques provided valuable insights into individual variables and their relationships. Comparing results helped highlight patterns, differences, and similarities, supporting better data-driven decision-making in healthcare contexts.

ASSIGNMENT NO. 10

TITLE

Case study to scale and improve BI systems.

PROBLEM STATEMET

To explore strategies for scaling and improving Business Intelligence (BI) systems through a detailed case study analysis.

OBJECTIVE

- To investigate the key challenges and performance bottlenecks in existing Business Intelligence (BI) systems.
- To analyze real-world case studies of organizations that have scaled and enhanced their BI systems effectively.
- To evaluate the impact of technological advancements and architectural strategies on BI system scalability and efficiency.
- To propose evidence-based recommendations for optimizing and expanding BI systems to support data-driven decision-making.

PLATFORM REQUIRED

- **Operating System:** Windows or Linux
- **Software/Tools:** BI Tool of choice (e.g., Tableau, Power BI, Looker)

THEORY

Case Study Overview:

- **Company Profile:** Choose a company (real or hypothetical) that has an existing BI system in place.
- **Current BI System:** Describe the existing BI architecture, tools, and processes.
- **Challenges:** Identify the challenges the company faces in scaling and improving its BI system.

Experiment Steps:

Part 1: Understanding the Current BI System

1. **System Architecture Documentation:**
    - Document the current BI system architecture, including data sources, ETL processes, data warehousing, and reporting tools.
    - Create a diagram to visualize the flow of data within the BI system.
2. **Performance Analysis:**
    - Evaluate the current performance of the BI system. Consider factors such as query performance, data refresh rates, and user satisfaction.

- Identify bottlenecks in the system, such as slow queries, outdated data, or limited scalability.

3. **User Requirements Gathering:**
   - Conduct interviews or surveys with BI system users (e.g., analysts, managers) to understand their needs and pain points.
   - Summarize the feedback and identify key areas for improvement.

Part 2: Identifying Scaling and Improvement Strategies

1. **Scalability Analysis:**
   - Evaluate the current BI system's ability to scale in terms of data volume, user load, and complexity of queries.
   - Research and propose strategies to scale the BI system, such as moving to a cloud-based data warehouse, optimizing ETL processes, or using advanced indexing techniques.
2. **Improvement Strategies:**
   - Identify strategies to improve the BI system, focusing on performance, data quality, and user experience.
   - Consider implementing data caching, indexing, query optimization, or upgrading the BI tool.
   - Explore the use of advanced analytics, AI/ML integration, or real-time data processing to enhance the system's capabilities.

Part 3: Implementation Plan

1. **Proposal Development:**
   - Develop a detailed proposal for scaling and improving the BI system, including the following components:
     - **Architecture Redesign:** Suggest modifications to the current architecture for better scalability and performance.
     - **Technology Stack:** Recommend upgrades or changes to the technology stack (e.g., migrating to a cloud-based data warehouse).

- **Process Improvements:** Outline improvements to ETL processes, data governance, and user training.
- **Cost-Benefit Analysis:** Provide a cost-benefit analysis of the proposed changes.

2. **Implementation Roadmap:**
   - Create a roadmap for implementing the proposed changes, including timelines, milestones, and key deliverables.
   - Identify potential risks and mitigation strategies.
3. **Pilot Testing:**
   - Plan and execute a pilot test of the proposed improvements with a subset of users.
   - Collect feedback and measure performance improvements.

Part 4: Evaluation and Reporting

1. **Performance Evaluation:**
   - Evaluate the performance of the improved BI system based on metrics such as query speed, user satisfaction, and data accuracy.
   - Compare the results with the baseline performance before the improvements.
2. **User Feedback:**
   - Gather feedback from users after the implementation of the improvements.
   - Assess whether the changes have addressed the initial challenges and improved the overall user experience.
3. **Final Report:**
   - Prepare a comprehensive report summarizing the case study, including:
     - Description of the original BI system and identified challenges.
     - Proposed strategies for scaling and improving the system.
     - Implementation process and results of the pilot test.
     - Final evaluation and recommendations for further improvements.

QUESTIONS

1. What are the common technical and organizational challenges that limit the scalability and performance of existing BI systems?

2. How have different organizations successfully scaled and optimized their BI systems, and what strategies contributed to their success?

3. What technological tools, architectures, or best practices are most effective in enhancing the efficiency and scalability of BI systems?

CONCLUSION

This study concludes that scaling and improving Business Intelligence (BI) systems requires a combination of advanced technologies, strategic planning, and best practices. Case study analysis reveals that cloud adoption, real-time analytics, and user-focused design significantly enhance BI performance and decision-making capabilities.

11. CBS1: Correlation Analysis

TITLE

Correlation Analysis for Identifying Relationships Between Variables in a Dataset.

PROBLEM STATEMET

To understand and perform correlation analysis on a dataset to identify and interpret the relationships between variables.

OBJECTIVE

- To understand the concept and significance of correlation in statistical analysis.
- To perform correlation analysis on a selected dataset using appropriate statistical techniques.
- To identify and measure the strength and direction of relationships between numerical variables.
- To interpret the results of the correlation analysis to draw meaningful insights from the data.

PLATFORM REQUIRED

- **Operating System:** Windows or Linux
- **Software/Tools:** Anaconda, Spyder, Jypter Notebook, Google Colab, Python 3.x

  Text Editor (such as Visual Studio Code, Sublime Text, or any IDE)

THEORY

Datasets:

Choose a dataset with multiple numerical variables for correlation analysis. Example datasets:

- **Iris Dataset**
- **Boston Housing Dataset**
- **Any dataset from Kaggle** with a good mix of continuous variables.

Experiment Steps:

Part 1: Introduction to Correlation

1. **Understanding Correlation:**
   - Briefly explain what correlation is, including the difference between positive, negative, and no correlation.
   - Introduce the correlation coefficient (Pearson's r) and explain its range (-1 to 1).
2. **Types of Correlation:**
   - Discuss different types of correlation:
     - **Pearson correlation:** Measures linear relationship between two continuous variables.
     - **Spearman rank correlation:** Measures the monotonic relationship between two ranked variables.
     - **Kendall tau correlation:** Measures the strength of association between two variables.

Part 2: Data Collection and Preparation

1. **Load the Dataset:**
   - Load the selected dataset into your working environment (Jupyter Notebook or RStudio).
2. **Data Cleaning:**
   - Inspect the dataset for missing values, outliers, and inconsistent data.
   - Handle missing values appropriately (e.g., imputation or removal).
   - Ensure that the data is ready for correlation analysis by checking the types of variables.
3. **Exploratory Data Analysis (EDA):**
   - Perform basic EDA to understand the structure of the data.
   - Generate summary statistics and initial visualizations (e.g., histograms, box plots) to get an overview of the data distribution.

Part 3: Performing Correlation Analysis

1. **Calculate Correlation Matrix:**

- Compute the correlation matrix for the numerical variables in the dataset using:
  - **Python:** pandas.DataFrame.corr() for Pearson correlation by default.
  - **R:** cor() function for Pearson correlation by default.
- If needed, calculate Spearman or Kendall correlations by specifying the method.

2. **Visualizing the Correlation Matrix:**
   - Use a heatmap to visualize the correlation matrix:
     - **Python:** Use seaborn.heatmap() to create a heatmap.
     - **R:** Use ggplot2 or the corrplot package for visualization.
   - Interpret the heatmap by identifying strong positive and negative correlations.
3. **Pairwise Scatter Plots:**
   - Create scatter plots for pairs of variables with significant correlations to visualize the relationships.
   - Assess whether the relationships appear linear or nonlinear.
   - Use pair plots (e.g., seaborn.pairplot() in Python or GGally::ggpairs() in R) to visualize relationships across multiple variables simultaneously.
4. **Significance Testing:**
   - Perform significance testing to determine whether the observed correlations are statistically significant.
   - **Python:** Use scipy.stats.pearsonr() for Pearson correlation or scipy.stats.spearmanr() for Spearman correlation.
   - **R:** Use cor.test() to test for the significance of the correlation.

Part 4: Interpretation and Reporting

1. **Interpret the Results:**
   - Discuss the strength and direction of the correlations found in the dataset.
   - Identify which variables are strongly correlated and consider the implications of these relationships.
   - Discuss any surprising or unexpected correlations.

2. **Reporting:**
   - Prepare a detailed report that includes:
     - An introduction to the dataset and objectives of the correlation analysis.
     - A description of the methodology used to calculate and visualize correlations.
     - A discussion of the findings, including key correlations and their potential implications.
     - Any limitations of the analysis or potential sources of bias.

Step 1: Load the Dataset and Required Libraries

```
# Import necessary libraries

import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

import numpy as np

from scipy.stats import pearsonr, spearmanr

# Load the Iris dataset

# You can load any dataset by replacing the file path or using pd.read_csv() for your dataset.

from sklearn.datasets import load_iris

# Load Iris dataset

data = load_iris()

df = pd.DataFrame(data.data, columns=data.feature_names)
```

```
# Display the first few rows of the dataset

df.head()
```

Step 2: Data Exploration and Cleaning

```
# Check for missing values

print("Missing values in each column:\n", df.isnull().sum())

# Basic statistical summary

print("Statistical summary:\n", df.describe())
```

Step 3: Compute Correlation Matrix

```
# Compute Pearson correlation matrix

correlation_matrix = df.corr()

# Display the correlation matrix

print("Correlation matrix:\n", correlation_matrix)
```

Step 4: Visualize the Correlation Matrix

```
# Visualize the correlation matrix using a heatmap

plt.figure(figsize=(10, 8))

sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', linewidths=0.5)

plt.title('Correlation Matrix Heatmap')

plt.show()
```

Step 5: Pairwise Scatter Plots

```
# Pairwise scatter plots to visualize relationships
```

```
sns.pairplot(df)

plt.suptitle('Pairwise Scatter Plots', y=1.02)

plt.show()
```

Step 6: Significance Testing

```
# Calculate Pearson and Spearman correlation with significance testing

for column1 in df.columns:

    for column2 in df.columns:

        if column1 != column2:

            pearson_corr, pearson_p = pearsonr(df[column1], df[column2])

            spearman_corr, spearman_p = spearmanr(df[column1], df[column2])

                print(f"Pearson correlation between {column1} and {column2}: {pearson_corr:.2f} (p-value: {pearson_p:.5f})")

                print(f"Spearman correlation between {column1} and {column2}: {spearman_corr:.2f} (p-value: {spearman_p:.5f})\n")
```

Step 7: Interpretation and Conclusion

- **Interpretation**:
    - After running the code, interpret the correlation matrix, heatmap, and pairwise scatter plots.
    - Analyze the strength and direction of correlations and discuss any interesting findings.

QUESTIONS

1. What is the role of correlation analysis in understanding relationships between variables in a dataset?
2. Which variables in the dataset show strong, weak, or no correlation, and what does this imply?
3. How can the results of correlation analysis be interpreted to support data-driven decision-making or further analysis?

CONCLUSION

Summarize the key findings, such as which variables are strongly correlated, and how this information might be useful for further analysis or modeling.

12. CBS2: Machine Learning Techniques for Data Science

TITLE

Implementation and Analysis of Core Machine Learning Techniques Using Python.

PROBLEM STATEMET

To implement and understand various machine learning (ML) techniques commonly used in data science, including regression, classification, and clustering, using Python.

OBJECTIVE

- To understand the fundamental concepts and applications of regression, classification, and clustering techniques in machine learning.
- To implement various ML algorithms using Python libraries such as Scikit-learn, Pandas, and Matplotlib.
- To evaluate and compare the performance of different machine learning models on real-world datasets.

PLATFORM REQUIRED

- **Operating System:** Windows or Linux
- **Software/Tools:** Anaconda, Spyder, Jypter Notebook, Google Colab, Python 3.x

  Text Editor (such as Visual Studio Code, Sublime Text, or any IDE)

THEORY

Datasets:

- **Iris Dataset**: For classification.
- **Boston Housing Dataset**: For regression.
- **Mall Customers Dataset**: For clustering.
- **Or any relevant dataset** based on the ML task.

Experiment Steps:

Part 1: Data Collection and Preprocessing

1. **Load the Dataset:**
   - Load the dataset into your working environment using pandas.
   - Example code for loading a dataset:

```
import pandas as pd

from sklearn.datasets import load_iris
```

```
# Load the Iris dataset

data = load_iris()

df = pd.DataFrame(data.data, columns=data.feature_names)

df['target'] = data.target
```

Part 2: Implementing Machine Learning Techniques

1. Regression (e.g., Linear Regression):

- **Objective**: Predict a continuous target variable.
- **Dataset**: Use the Boston Housing dataset for this example.

Steps:

1. Load the Boston Housing dataset.
2. Train a Linear Regression model.
3. Evaluate the model using metrics like Mean Squared Error (MSE) and R-squared.

```
from sklearn.datasets import load_boston

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error, r2_score

# Load dataset

boston = load_boston()

X = pd.DataFrame(boston.data, columns=boston.feature_names)

y = boston.target

# Split the data

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
# Train the model

model = LinearRegression()

model.fit(X_train, y_train)

# Predictions

y_pred = model.predict(X_test)

# Evaluation

print("MSE:", mean_squared_error(y_test, y_pred))

print("R-squared:", r2_score(y_test, y_pred))
```

2. **Classification (e.g., Logistic Regression):**

- **Objective**: Predict a categorical target variable.
- **Dataset**: Use the Iris dataset for classification.

Steps:

1. Load the Iris dataset.
2. Train a Logistic Regression model.
3. Evaluate the model using metrics like accuracy, confusion matrix, and classification report.

```
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load dataset

iris = load_iris()

X = pd.DataFrame(iris.data, columns=iris.feature_names)

y = iris.target
```

```
# Split the data

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Train the model

model = LogisticRegression(max_iter=200)

model.fit(X_train, y_train)

# Predictions

y_pred = model.predict(X_test)

# Evaluation

print("Accuracy:", accuracy_score(y_test, y_pred))

print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

print("Classification Report:\n", classification_report(y_test, y_pred))
```

3. Clustering (e.g., K-Means Clustering):

- **Objective**: Group similar data points into clusters.
- **Dataset**: Use a dataset like the Mall Customers dataset.

Steps:

1. Load the dataset.
2. Perform K-Means clustering.
3. Visualize the clusters.

```
from sklearn.cluster import KMeans

import matplotlib.pyplot as plt

# Example data: generate some random 2D data points
```

```
from sklearn.datasets import make_blobs

X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=0)

# Train the model

kmeans = KMeans(n_clusters=4)

kmeans.fit(X)

# Predict clusters

y_kmeans = kmeans.predict(X)

# Visualize the clusters

plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, cmap='viridis')

centers = kmeans.cluster_centers_

plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, alpha=0.75)

plt.title('K-Means Clustering')

plt.show()
```

Part 3: Advanced Techniques and Model Fine-tuning

1. **Hyperparameter Tuning:**
   - Use GridSearchCV or RandomizedSearchCV to find the best hyperparameters for your model.

```
from sklearn.model_selection import GridSearchCV

# Example with Logistic Regression

param_grid = {'C': [0.1, 1, 10], 'solver': ['liblinear']}
```

```
grid = GridSearchCV(LogisticRegression(), param_grid, refit=True, verbose=0)

grid.fit(X_train, y_train)

# Best parameters

print("Best Parameters:", grid.best_params_)
```

2. **Cross-Validation:**

- Implement cross-validation to ensure the model's performance is consistent across different data subsets.

```
from sklearn.model_selection import cross_val_score

# Example with Linear Regression

scores = cross_val_score(LinearRegression(), X, y, cv=5)

print("Cross-Validation Scores:", scores)

print("Mean CV Score:", scores.mean())
```

<u>Part 4: Model Evaluation and Reporting</u>

1. **Model Evaluation:**
   - Evaluate the performance of each model using appropriate metrics (MSE, accuracy, etc.).
   - Compare different models and discuss their performance.
2. **Reporting:**
   - Document the entire process, including data preparation, model implementation, and evaluation.
   - Discuss the strengths and weaknesses of each technique used.

QUESTIONS

1. How do regression, classification, and clustering techniques differ in terms of their purpose and application in data science?

2. How can Python libraries be used to implement and visualize machine learning models effectively?

3. What evaluation metrics can be used to assess the performance of regression, classification, and clustering models?

CONCLUSION

In conclusion, the implementation of various machine learning techniques—regression, classification, and clustering—using Python enhances practical understanding of core data science methods. This hands-on approach helps in building predictive and analytical models, enabling informed decision-making based on data-driven insights.